

Rapport projet ARN

1-Présentation Générale de l'Arbre Rouge-Noir (ARN)

L'arbre rouge-noir (ARN) est une structure de données auto-équilibrée utilisée principalement dans les systèmes nécessitant des opérations efficaces de recherche, insertion et suppression. Contrairement aux arbres binaires de recherche classiques (ABR), les arbres rouge-noir garantissent que l'arbre reste équilibré, ce qui permet d'effectuer ces opérations en temps logarithmique, même dans les cas les plus défavorables.

1. Principe de l'Arbre Rouge-Noir

Un arbre rouge-noir est un arbre binaire de recherche où chaque nœud possède une couleur (rouge ou noir). Il suit un ensemble de propriétés qui garantissent l'équilibre de l'arbre :

- **Propriété 1** : Chaque nœud est soit rouge, soit noir.
- **Propriété 2** : La racine est toujours noire.
- **Propriété 3** : Les feuilles (nœuds nuls) sont toujours noirs.
- **Propriété 4** : Si un nœud est rouge, alors ses enfants sont nécessairement noirs (pas de deux nœuds rouges consécutifs).
- **Propriété 5** : Pour chaque nœud, tous les chemins simples de ce nœud jusqu'aux feuilles doivent contenir le même nombre de nœuds noirs.

Ces règles garantissent que l'arbre reste équilibré et que la hauteur de l'arbre est toujours proportionnelle à $\log(n)$, où n est le nombre de nœuds de l'arbre.

2. Méthodes Utilisées et Fonctionnement

Les principales méthodes utilisées dans un arbre rouge-noir sont celles permettant l'ajout, la suppression, la recherche de nœuds, ainsi que l'équilibrage automatique de l'arbre à chaque modification.

2.1 Insertion

- Lors de l'insertion d'un nouvel élément, le nœud est initialement coloré en **rouge**.
- Si la propriété des nœuds rouges consécutifs (Propriété 4) est violée, un **rééquilibrage** (rotation et recolorisation) est effectué pour restaurer l'équilibre de l'arbre.
- Des rotations **à gauche** ou **à droite** sont réalisées pour maintenir l'ordre des éléments tout en assurant les propriétés de couleur et de structure de l'arbre.
- Après insertion, l'arbre peut nécessiter une série de rotations et recolorisations pour s'assurer que les règles sont respectées, en particulier la règle interdisant les nœuds rouges adjacents.

2.2 Suppression

- La suppression d'un nœud implique aussi un rééquilibrage, car la suppression peut perturber l'équilibre de l'arbre.
- Après suppression, un nœud noir doit être remplacé par un nœud noir (si possible), ce qui peut entraîner une **rotation** pour maintenir la couleur noire de tous les chemins depuis la racine.

- Des ajustements similaires à ceux effectués lors de l'insertion sont nécessaires pour restaurer les propriétés de l'arbre après suppression.

2.3 Recherche

La recherche dans un arbre rouge-noir fonctionne de manière similaire à celle d'un arbre binaire de recherche classique. On descend dans l'arbre en comparant la clé du nœud courant avec celle que l'on recherche. Si la clé est plus petite, on va dans le sous-arbre gauche, sinon on va dans le sous-arbre droit.

Cependant, les opérations de recherche dans un arbre rouge-noir sont assurées de se terminer en temps **logarithmique**, grâce aux propriétés de l'arbre qui maintiennent un certain équilibre.

2.4 Rééquilibrage (Rotations et Recolorisation)

Le rééquilibrage d'un arbre rouge-noir se fait principalement via deux mécanismes :

- **Rotation à gauche** : Utilisée pour réorganiser l'arbre lorsque les sous-arbres sont déséquilibrés par rapport à un nœud donné.
- **Rotation à droite** : Similaire à la rotation à gauche, mais utilisée pour corriger un déséquilibre dans la direction opposée.
- **Recolorisation** : Modifier la couleur des nœuds pour rétablir les règles de l'arbre (par exemple, changer un nœud rouge en noir et vice versa).

Ces opérations sont essentielles pour maintenir les propriétés de l'arbre rouge-noir après chaque insertion ou suppression.

3. Temps d'Exécution

Les opérations de base dans un arbre rouge-noir (insertion, suppression et recherche) ont toutes une complexité **$O(\log n)$** , où n est le nombre de nœuds dans l'arbre. Cette garantie est rendue possible par le mécanisme de rééquilibrage, qui permet de garder l'arbre toujours bien équilibré et de limiter la hauteur de l'arbre à environ $2 * \log n$.

4. Applications Pratiques

Les arbres rouge-noir sont utilisés dans de nombreuses applications où il est essentiel de maintenir un équilibre strict tout en permettant des recherches rapides. Par exemple :

- **Structures de données standard** : L'implémentation de l'interface `TreeMap` de Java utilise un arbre rouge-noir.
- **Systèmes de bases de données** : Pour stocker des index dans des bases de données, où les requêtes de recherche, d'insertion et de suppression doivent être rapides.
- **Systèmes de fichiers** : Certains systèmes de fichiers utilisent des arbres rouge-noir pour gérer l'organisation des fichiers.

Conclusion

L'arbre rouge-noir est une structure robuste permettant des opérations efficaces sur des ensembles de données. Grâce à son mécanisme d'équilibrage automatique, il permet des recherches rapides même dans les pires cas, ce qui le rend idéal pour les applications nécessitant des performances constantes pour des ensembles de données dynamiques. Son implémentation repose sur des

rotations et recolorisations, qui assurent l'intégrité et l'équilibre de l'arbre après chaque modification.

2- Comparaison des Performances

VV

Structure	Complexité Insertion	Complexité Recherche	Complexité Suppression	Avantages	Inconvénients
ABR classique	$O(h)$ (pire cas $O(n)$)	$O(h)$ (pire cas $O(n)$)	$O(h)$ (pire cas $O(n)$)	Simple, rapide si équilibré	Temps de fonctionnement dégradé si l'arbre est déséquilibré
ARN	$O(\log n)$	$O(\log n)$	$O(\log n)$	Équilibré, garanti en $O(\log n)$	Plus complexe à implémenter, rotations et recolorisations
Structures linéaires (tableaux, listes)	$O(n)$ (tableaux), $O(1)$ (listes liées)	$O(n)$	$O(n)$	Simple à comprendre et implémenter	Recherche et modification lentes dans le pire cas

3- Choix spécifiques dans l'implantation de l'Arbre Rouge-Noir (ARN)

1. Utilisation de la couleur des nœuds (rouge/noir) :

- La couleur des nœuds est un aspect central pour maintenir l'équilibre de l'arbre et garantir des performances logarithmiques. Les nœuds sont initialement rouges, et après chaque insertion, des corrections sont effectuées pour maintenir les propriétés des arbres rouges-noirs, notamment via les rotations et recolorisations.

2. Gestion de la racine :

- La racine de l'arbre est systématiquement colorée en **noir** lors de l'insertion pour respecter la règle fondamentale d'un arbre rouge-noir : la racine doit toujours être noire. Cela permet de garantir la stabilité de l'arbre après chaque modification.

3. Insertion et correction après insertion :

- Lors de l'insertion, après l'ajout d'un nœud, une méthode `corrigerInsertion` est appelée pour ajuster l'arbre en fonction des règles des arbres rouges-noirs. Cette méthode gère les cas d'équilibrage via des rotations et recolorations pour maintenir la hauteur logarithmique.

4. Rotations gauche et droite :

- Les rotations sont utilisées lors de l'insertion et de la suppression pour rétablir l'équilibre. Ces opérations sont essentielles pour garantir que la hauteur de l'arbre reste proportionnelle à $\log(n)$, ce qui est crucial pour maintenir les performances dans le pire cas.

5. Suppression et correction après suppression :

- Lors de la suppression, un mécanisme similaire à celui de l'insertion est utilisé pour ajuster l'arbre après la suppression d'un nœud. Le processus de correction

(supprimerCorrection) garantit que l'arbre reste équilibré, en prenant en compte les couleurs et en appliquant les rotations nécessaires.

6. Gestion de l'itération :

- Un itérateur `ABRIterator` est fourni pour parcourir l'arbre dans l'ordre croissant des éléments. Cela permet d'itérer efficacement sur les éléments de l'arbre, en utilisant les propriétés de l'arbre binaire de recherche pour déterminer le successeur d'un nœud.

7. Utilisation d'un comparateur optionnel :

- Le comparateur (`Comparator<? super E> cmp`) permet une flexibilité dans la gestion des ordres des éléments dans l'arbre. Si aucun comparateur n'est fourni, les éléments doivent être comparables entre eux. Cela permet d'adapter l'implémentation à différents types de données.

En utilisant mes résultats prétendants on peut remarquer que :

Complexité : Les arbres rouges-noirs (ARN) sont plus efficaces dans tous les cas, avec des complexités de temps de **$O(\log n)$** pour la construction, l'insertion et la recherche, contrairement aux ABR classiques qui peuvent devenir linéaires dans le pire cas.

Temps d'exécution : L'ARN est beaucoup plus stable en termes de performances. Les ABR, bien que performants dans le cas moyen, peuvent devenir beaucoup moins efficaces si les éléments sont insérés dans un ordre défavorable (ordre trié).

Hauteur de l'arbre : Les ARN maintiennent une hauteur optimale de **$O(\log n)$** , garantissant de bonnes performances à tout moment, tandis que les ABR peuvent devenir dégénérés, avec une hauteur allant jusqu'à **$O(n)$** .

En conclusion, les **arbres rouge-noir** sont préférables dans des contextes où la performance dans tous les cas (y compris le pire cas) est importante, tandis que les **arbres binaires de recherche classiques** peuvent être utilisés dans des cas où l'ordre d'insertion est contrôlé ou garanti d'être aléatoire.

Méthodologie d'Extraction et Analyse de la Complexité

1. Extraction des Données de Performance

L'extraction des données de performance a été réalisée via l'exécution d'une classe Java unifiée (EtudeComplexite.java). Le temps d'exécution des opérations (construction et recherche) a été mesuré avec précision en nanosecondes en utilisant la fonction `System.nanoTime()`.

Deux scénarios critiques ont été testés pour des tailles d'arbres allant jusqu'à $N=100,000$:

1. **Cas Moyen** : Insertion des éléments dans un ordre aléatoire.
2. **Pire Cas** : Insertion des éléments dans un ordre trié,

Les résultats bruts ont été capturés depuis la console et exportés vers le fichier `resultats.csv`.

2. Comparaison des Performances

L'analyse des temps d'exécution (en millisecondes) démontre l'avantage fondamental de l'Arbre Rouge-Noir (ARN) sur l'Arbre Binaire de Recherche (ABR).

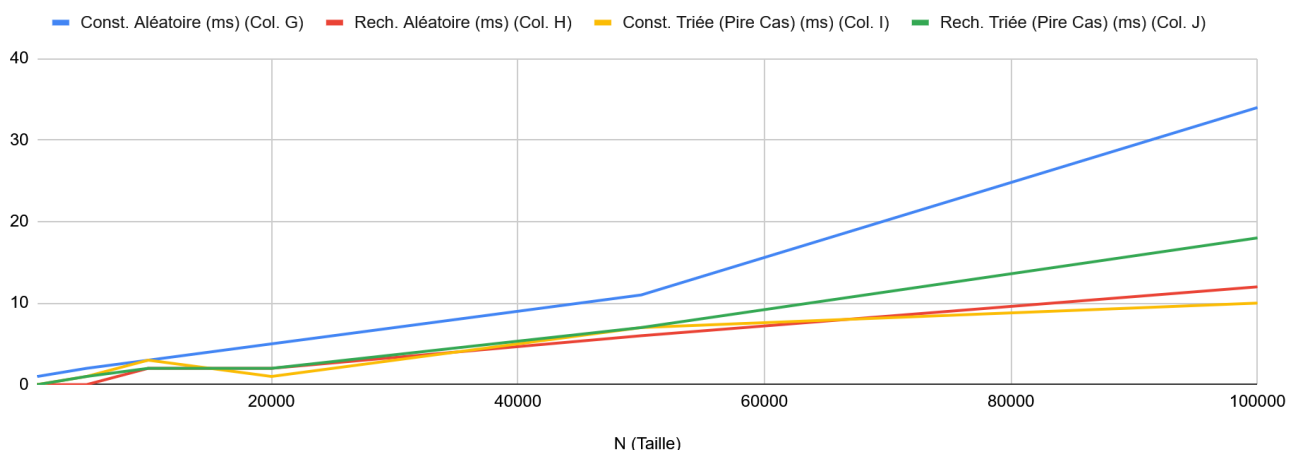
Arbre Rouge-Noir (ARN)

Le graphique de performance de l'ARN montre une courbe de croissance très faible, tendant vers l'axe des abscisses. L'augmentation des temps d'exécution est strictement proportionnelle à $\log N$.

Ceci démontre que les opérations de rééquilibrage (rotations et recoloriages) sont suffisamment rapides pour **empêcher l'arbre de se déséquilibrer**, assurant ainsi que la hauteur de l'arbre reste toujours $O(\log N)$. L'ARN est donc la structure de données préférée pour garantir une performance optimale, stable et bornée, quel que soit l'ordre des données

Les résultats mesurés le confirment : même dans le pire cas d'insertion triée à $N=100,000$, les temps de construction et de recherche restent très faibles (moins de 34 ms), ne montrant qu'une croissance marginale.

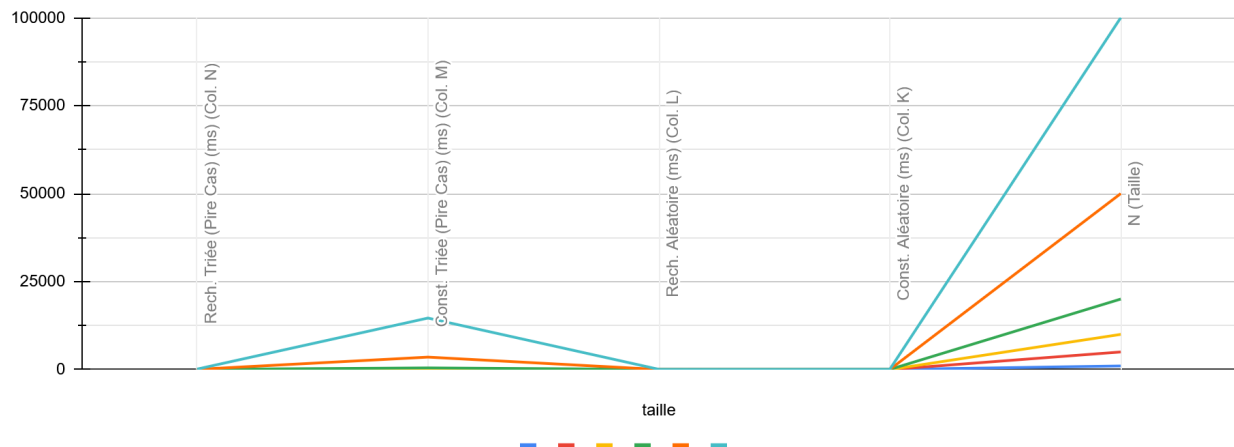
Représentation du temps d'exécution en fonction de la taille des données des ARN pour les 4 types de tris



Arbre Binaire de Recherche (ABR)

L'étude des performances de l'ABR révèle une forte dépendance à l'ordre d'insertion des données

Représentation du temps d'exécution en fonction de la taille des données des ABR pour les 4 types de tris



Le Cas Moyen (Insertion Aléatoire)

L'ABR fonctionne bien lorsque les éléments sont insérés de manière aléatoire. Dans ce cas, l'arbre reste relativement équilibré, et les opérations se font en temps logarithmique

- À $N=100,000$, la construction de l'ABR aléatoire ne prend que **37 millisecondes**, prouvant son efficacité dans des conditions optimales.

Le Pire Cas : L'Effondrement

Le problème survient dans le scénario du **pire cas (insertion triée)**, où l'ABR dégénère en une simple liste chaînée.

- Cette dégénérescence force la complexité de la construction à devenir **quadratique ($O(N^2)$)**.
- L'impact est dramatique : le temps de construction passe de 441 ms pour $N=20,000$ à **14 605 millisecondes (environ 14,6 secondes)** pour $N=100,000$.

Cette croissance spectaculaire du temps d'exécution prouve que l'ABR est **fragile et inutilisable** pour de grands volumes de données si l'ordre d'insertion n'est pas strictement aléatoire.

- La complexité de la construction explose en **quadratique ($O(N^2)$)**. Les temps mesurés illustrent cet effondrement : le temps de construction passe de 441 ms pour $N=20,000$ à **plus de 14 secondes (14 605 ms)** pour $N=100,000$.
- La complexité de la recherche dans cet arbre dégénéré devient **linéaire ($O(N)$)**.

Conclusion sur les Performances

L'étude valide que l'**ARN est la structure de données supérieure** lorsqu'une garantie de performance est requise. Là où l'ABR est vulnérable aux données non aléatoires (entraînant un effondrement quadratique), l'ARN maintient une performance rapide et bornée en temps logarithmique pour toutes les situations.